Behaviour Trees and Recovery: Deciding What the Robot Does Next

Contents

1. Overview . . . 2
2. Choosing what to do next: the finite state machine . . . 4
3. Behaviour trees: a tick and three answers . . . 6
4. Sequence and fallback . . . 7
5. Decorators and the blackboard . . . 10
6. Nav2's default tree, tick by tick . . . 11
7. Recovery behaviours: when no path exists . . . 14
8. The goal checker and the progress checker . . . 20
9. Waypoint following . . . 22
10. How cars switch behaviours: Apollo and Autoware . . . 25
    - 10.1 Apollo: scenarios and stages . . . 25
    - 10.2 Autoware: scene modules in slots . . . 28
    - 10.3 Three ways compared . . . 29
11. Summary . . . 31
12. Sources . . . 32
13. Key terms . . . 34

Where this fits:

- **Builds on:** ROS 2 actions.
- **Leads to:** Controller server; Nav2, start to finish.

1. Overview

Key point: A behaviour tree decides what the robot does next by sending a tick from its root every 10 ms; each node answers success, failure or running, and sequences and fallbacks turn those answers into the next step. When the door is blocked, Nav2's default tree on `main` checks the path once a second, asks the planner again, and then tries clearing the costmaps, spinning and waiting in turn: the robot reaches the door at 15.60 s instead of 7.37 s, and aborts the goal after six recoveries if no path ever appears.

The robot can already drive round a person the map does not contain. A local planner such as TEB bends its trajectory round the person every 0.1 s, and Nav2's DWB controller reaches the door when the plan is remade every second. Both work only while a way to the door exists. When TEB's trajectory is unsafe it stops and reports a failure, and the controller server passes the failure on. Neither planner decides what happens next.

The problem of this Note, on numbers. The robot starts at rest at (2.0, 1.0) facing 30° and must stop on the door pose (3.6, 3.5) facing 90°. At 2.4 s a person stops in the gap between the table's corner and the wall, just left of the door, and a trolley stands in the only other way, between the table and the cabinet. Figure 1 shows the room at 3.01 s. The four gaps left are 0.30 m, 0.47 m, 0.11 m and 0.10 m wide, and the robot is 0.50 m wide, so no path to the door exists. At 8.0 s the person walks away. Between 3.01 s and 8.0 s something must decide:

- whether the path is still valid, and when to ask for a new one;
- what to try when the planner finds no path: clear the costmaps, turn, wait or back up, and in which order;
- when the goal counts as reached, when the robot counts as stuck, and when to abort the goal.

Nav2 stores its obstacle information in two costmaps, grid maps whose cells hold a cost, highest where an obstacle sits: a global one, searched by the planner for a path, and a local one round the robot, used by the controller (Nav2 docs, Environmental Representation; Nav2 `nav2_params.yaml`).

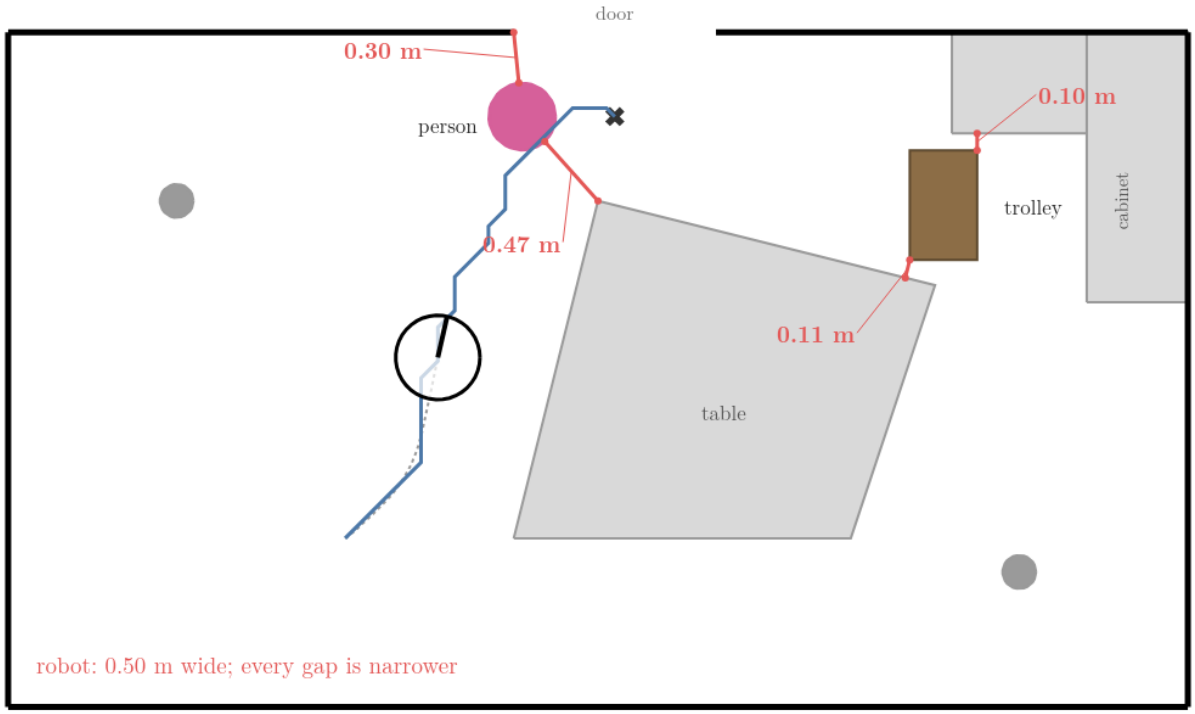


Figure 1: The blocked door at 3.01 s, in our run (our Python version, not Nav2 running). Black disk: the robot, 0.50 m wide, with its heading; dotted: its track since 0.0 s; blue: the path it was given at 0.01 s, which now runs through the person (pink). Brown: the trolley. Red: the four gaps, measured from the shapes: 0.30 m from the wall's end at the door to the person, 0.47 m from the person to the table's corner, 0.11 m from the trolley to the table and 0.10 m from the trolley to the cabinet. Every gap is narrower than the robot, so the door cannot be reached until the person leaves at 8.0 s

Deciding what to do next is the job of the behavioural layer (G-2912), the layer of the autonomy stack (G-3210) that picks the task to carry out now. This Note builds that layer in the form Nav2 uses, a behaviour tree, and answers the three questions on the blocked door. Nav2's default tree on `main` detects the blocked path 0.61 s after the person stops, at its next once-a-second check. The tree asks the planner for a path, clears the costmaps and asks again, then spins 1.57 rad and waits 5 s, asking again after each; six times the planner answers that no path exists. When the wait ends at 9.78 s the person has gone, a new path comes at 9.79 s, and the goal is accepted at 15.60 s. In the clear room the same tree reaches the door at 7.37 s.

What we ran. The runs in this Note are our own Python version of Nav2's default behaviour tree (`images/btkit.py`), written from its code on the branch `main`, read on 2026-10-12. Our version is not Nav2 running:

- the control nodes copy the code of BehaviorTree.CPP version 4, the library Nav2 uses, and of Nav2's `nav2_behavior_tree` package;

- the tree is Nav2's default on `main`, `navigate_to_pose_w_replanning_and_recovery.xml`, ticked every 10 ms; its five selector nodes, which only pick plug-ins by name, are drawn as one node;
- the servers the tree calls are simple stand-ins: the planner is A* on 0.1 m cells, not NavFn; the controller is a path follower that looks 0.4 m ahead and does not itself steer round obstacles, not DWB or MPPI, with Nav2's goal checker and progress checker; Spin, Wait and BackUp copy the behaviour server's rules and its bring-up values;
- the costmap is the room's exact shapes: clearing it erases the marks, and the next sensor reading marks again whatever is still there, so clearing cannot remove the person;
- a server's answer arrives one tick, 10 ms, after the request.

The sections follow the questions in order:

- why a fixed list of states and transitions grows hard to change (Section 2);
- the tree, its tick and its three answers (Section 3);
- sequence and fallback (Section 4), decorators and the blackboard (Section 5);
- Nav2's default tree on the drive with the person (Section 6) and its recoveries on the blocked door (Section 7);
- the goal checker and the progress checker (Section 8), waypoint following (Section 9);
- how Apollo and Autoware switch behaviours in cars (Section 10).

2. Choosing what to do next: the finite state machine

> **Key point:** A finite state machine names each task as a state and draws an arrow for every switch between tasks. Nav2's recoveries written this way need 12 transitions and a counter, and adding one task, docking when the battery is low, adds 7 more: one from each of the 6 states that can be active, and one back.

The opening's first need is a rule for what comes after each task: after planning, drive or recover; after a recovery, plan again; after six recoveries, abort.

The idea. The most direct way to write that rule is a flowchart: a box for each task, and an arrow for each way of moving from one box to another, labelled with the event that causes it. A model with a fixed set of states, one of which is active at a time, and transitions between them, each triggered by an event, is a **finite state machine** (FSM) (G-3299) (Colledanchise and Ögren §2.1). A turnstile is the everyday example: locked, a coin unlocks it; unlocked, a push locks it again.

On the room. Figure 2 writes the blocked-door problem as an FSM with the states Plan, Drive, Clear costmaps, Spin, Wait, Back up, Arrived and Aborted. Look at the arrows into and out of Plan: each recovery state needs an arrow in and an arrow back, and the arrow to take depends on how many plans have already failed, so the machine also needs a counter. Then look at the red dashed arrows: to add docking when the battery is low, every state that can be active needs a new arrow to Dock.

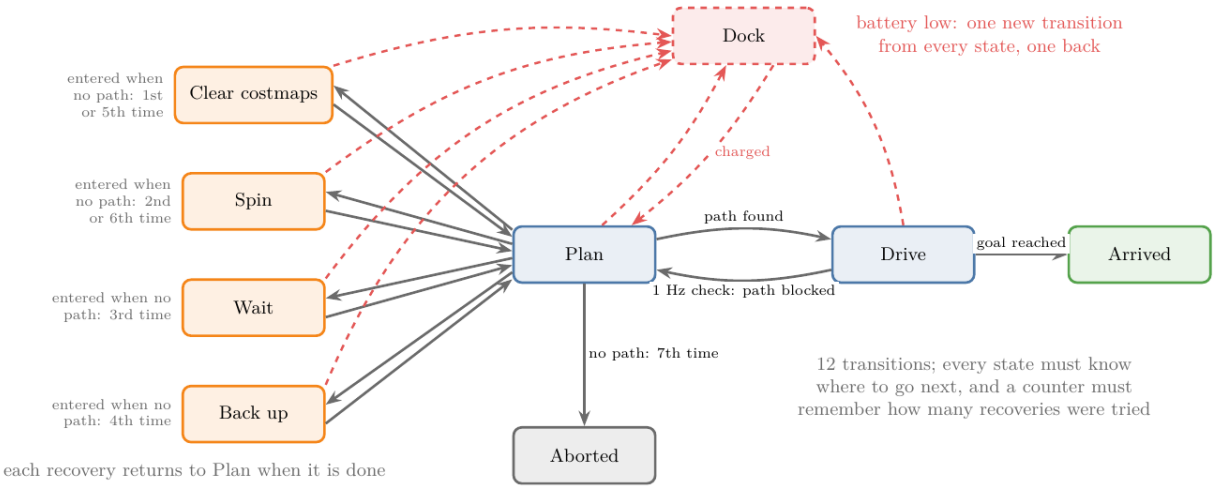


Figure 2: The room's navigation with Nav2's recoveries written as a finite state machine. Blue: planning and driving; orange: the four recoveries, each entered when the planner finds no path for the time shown and left back to Plan when it is done; green and grey: the two end states. Grey arrows: its 12 transitions. Red dashed: what docking on a low battery adds, one transition from each of the 6 states that can be active and one back to Plan

The count of transitions, from Figure 2 (notebook):

$$\text{transitions} = 6 + 2 + 4 \times 1$$

$$= 12$$

$$\text{with docking} = 12 + 6 + 1$$

$$= 19$$

The 6 are Plan's exits (Drive, four recoveries, Aborted), the 2 are Drive's (back to Plan, Arrived), and each recovery has one exit, back to Plan.

Why it grows. A transition is a one-way jump: the active state must name the state that comes next, for every event. Adding or removing one state therefore means editing every state that can reach it, and the more transitions a machine has to react quickly, the more edits each change needs (Colledanchise and Ögren §1.2). Nav2's documentation gives the same reason for not using an FSM: navigation as an FSM "may have dozens of states and hundreds of transitions" (Nav2 docs, Behavior Trees). ROS 1's navigation program, `move_base`, ran a fixed state machine of this kind (Nav2 start to finish).

The FSM answers what comes next, but at the price of arrows in every state. Next, a structure in which a task does not need to name the task that comes after it.

3. Behaviour trees: a tick and three answers

> **Key point:** In a behaviour tree, a task never chooses the next task: its parent asks it to run one step, a tick, and it answers success, failure or running. The parents above it decide what comes next, and in Nav2 the root is ticked every 10 ms.

Section 2's Plan state had to name six next states. The remedy is to let each task report only its outcome, and to place the choice of what comes next in the structure above the tasks.

The idea. A function in a program is called, runs and returns to the caller, which decides what to call next; the function does not jump to another part of the program. A behaviour tree arranges a robot's tasks in that way (Colledanchise and Ögren §1.2). The tasks are the leaves of a tree. The nodes above them, the control nodes, decide which leaf runs. Each leaf only reports one of three answers.

The standard terms. A **behaviour tree** (BT) (G-3300) is a tree whose leaves are tasks and checks and whose inner nodes decide, from the leaves' answers, which leaf runs next; it chooses the next task without any task naming another (Colledanchise and Ögren §1.3). A **tick** (G-3301) is the signal a parent sends to a child to run one step, starting from the root at a fixed rate; a node runs only while it receives ticks (Colledanchise and Ögren §1.3; BehaviorTree.CPP docs, "Introduction to BT basics"). The **return status** (G-3302) is the node's answer to a tick:

- **success:** the task is done, or the check holds;
- **failure:** the task cannot be done, or the check does not hold;
- **running:** the task needs more time, such as following a path for several seconds.

Two kinds of leaf are used:

- an **action node** (G-3303) does something, such as asking the planner for a path or turning in place, and answers success, failure or running;
- a **condition node** (G-3304) checks something, such as whether the path is still free, and answers only success or failure, never running.

A leaf that takes a long time, such as following a path for 10 s, answers running on every tick until it ends; its parent answers running too, and so on up to the root. In Nav2 most action nodes send a request to a server as a ROS 2 action (G-3227), a long job with feedback, and answer running until the server's result comes back. Nav2's documentation warns that a node of a tree is not a ROS 2 node (Nav2 docs, Nav2-specific nodes).

On the room. Figure 3 shows the node symbols and the smallest navigation tree: check whether the path is valid, else compute one, and then follow it. Read its first tick, at 0.00 s, in the order numbered. There is no path yet, so the condition fails; the tree then asks for a path, which takes a tick, so that action answers running; the parents above it answer running too.

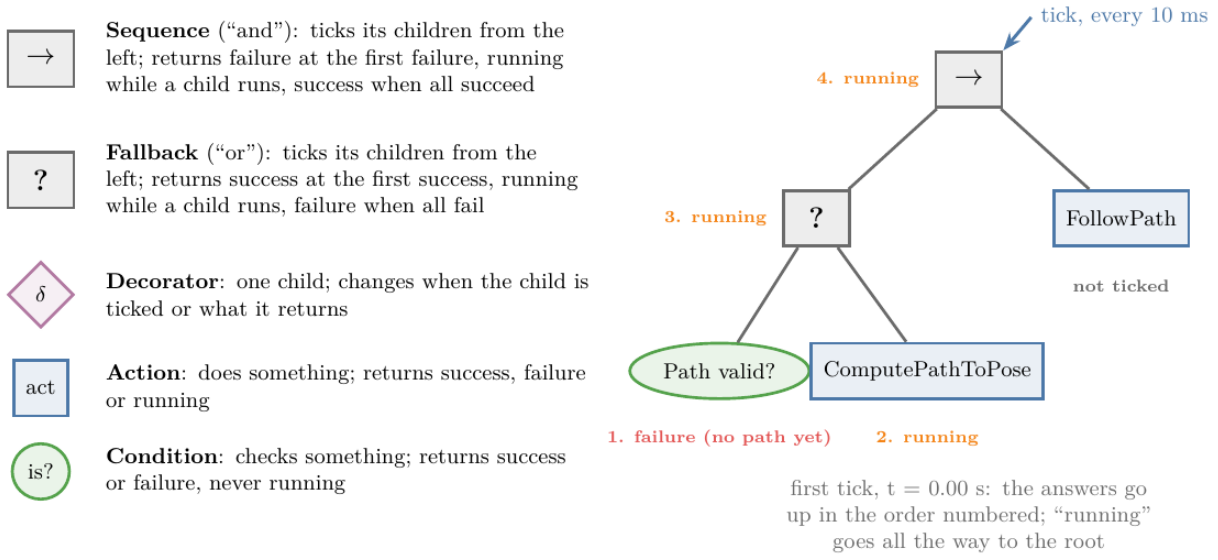


Figure 3: Left: the node kinds of a behaviour tree and their symbols. Right: the smallest navigation tree at its first tick, at 0.00 s in our run: a sequence (arrow) whose first child is a fallback (question mark) over the condition "Path valid?" (green ellipse) and the action ComputePathToPose (blue box), and whose second child is the action FollowPath. Coloured labels: the answers, in the order they go up the tree; FollowPath is not ticked in this tick

How often. The tick rate sets how fast the tree can detect a change. Nav2's tree navigator ticks the root every 10 ms (`bt_loop_duration: 10`, Nav2 `nav2_params.yaml`), so a run of 9.77 s takes 978 ticks (notebook). Most ticks leave every answer unchanged: they pass down to the running action and return running.

The tree reports and runs tasks, but nothing yet turns the answers into a choice. Next, the two control nodes that do.

4. Sequence and fallback

> **Key point:** A sequence ticks its children in order and stops at the first failure, an "and"; a fallback stops at the first success, an "or", and a condition placed first in a fallback means "check before you act". A plain sequence goes on from its running child, while Nav2's pipeline sequence ticks the children before it again, so its path check runs on every tick.

Section 3's answers say how each task went. The opening needs them combined: plan, then drive; and plan only when the path is not valid.

The idea. People combine steps in two ways. "Unlock the door, then open it" is an "and": if unlocking fails, opening is not tried. "Take the lift, or else the stairs" is an "or": the stairs are tried only if the lift fails.

The standard terms. A **sequence node** (G-3305) ticks its children from the left; it answers failure at the first child that fails, running while a child runs, and success only when every child has succeeded, so it runs steps that must all happen in order. A **fallback node** (G-3306), also called a selector, ticks its children from the left; it answers success at the first child that succeeds, running while a child runs, and failure only when every child has failed, so it tries options in order of preference (Colledanchise and Ögren §1.3, Table 1.1). Both are control nodes, written → and ? in Figure 3.

Check before you act. In Figure 3's tree, the fallback's first child is the condition "Path valid?". While the path is valid, the condition succeeds, the fallback succeeds at once, and the planner is not called. Only a failed check leads to ComputePathToPose. A condition placed first in a fallback is how a tree avoids repeating a task whose goal already holds (Colledanchise and Ögren §3.1).

Going on, or checking again. What happens on the next tick decides whether the tree ever detects the person. In BehaviorTree.CPP's plain `Sequence`, a child that has succeeded is not ticked again while a later child is running: the next tick goes straight to the running child (BehaviorTree.CPP docs, "Sequences"). Nav2's `PipelineSequence` instead ticks every child before the running one again on each tick (Nav2 docs, Nav2-specific nodes).

Figure 4 runs the smallest tree under both parents on the drive of RO-058, in which a person steps onto the route at 2.4 s but leaves a way round, so the planner can always find a path and only the checking differs. Watch the left tree from 0.02 s: only FollowPath is ticked. Watch the right tree at 2.40 s: "Path valid?" fails in the tick in which the person appears, and the new path arrives at 2.41 s.



Figure 4: Animation: the smallest navigation tree, tick by tick, under two parents (our Python version, not Nav2 running). Left: a plain Sequence, which goes on from its running child. Right: Nav2's PipelineSequence, which ticks the children before the running one again. Top: the room, with the path (blue), the robot's track (dotted) and, from 2.4 s, the person (pink). Bottom: each box is coloured by what the node answered in that tick: amber running, green success, red failure, white not ticked. The last frame, at 3.10 s, holds: left, the old path still runs through the person; right, the robot follows a path round the person

The counts, from the same two runs (notebook):

- plain `Sequence`: "Path valid?" ticked once, at 0.00 s; one plan, at 0.01 s;
- `PipelineSequence`: "Path valid?" ticked 309 times in 3.10 s; a second plan at 2.41 s, 0.01 s after the person stepped in.

The plain sequence never asks again whether its path is valid, so only the controller is left to stop the robot. The pipeline asks on every tick, which sends the planner a new request whenever the path is blocked. Next, how the default tree limits that checking to once a second, and where the nodes keep the path they share.

5. Decorators and the blackboard

> **Key point:** A decorator has one child and changes when that child is ticked or what its answer is; Nav2 uses an inverter and a 1 Hz rate controller. The blackboard is the tree's shared store of named entries, such as the goal, the path and the planner's error code, through which nodes pass data without knowing one another.

Section 4's pipeline checks the path 100 times a second. The opening's robot needs that check only once a second, and needs the planner's path and error code passed to the nodes that use them.

The idea. A timer on a lamp does not change the lamp; it decides when the lamp may be switched. The word "not" does not change a statement; it turns a true answer into a false one. A **decorator node** (G-3307) is a control node with exactly one child that changes when the child is ticked or what its answer becomes, such as flipping success and failure or letting a tick through only once a second (Colledanchise and Ögren §1.3; BehaviorTree.CPP docs, "Introduction to BT basics"). Figure 3 draws it as a rhombus.

On the room. Nav2's default tree uses two decorators (Nav2 default tree), shown in Figure 5, top:

- `Inverter` over the condition `GlobalUpdatedGoal?`: the condition succeeds only when a new goal has arrived; flipped, its failure becomes success, so the path check can go on when the goal is unchanged;
- `RateController` at 1 Hz over the planning part of the tree: it ticks that part at 0.00 s and then 1 s after each success, at 1.01, 2.01, 3.01 and 4.02 s in the run with the person; between those ticks it answers running without ticking it (notebook).

The rate controller's period is the reason the default tree is slower to detect the person than Figure 4's pipeline: the person steps in at 2.40 s and the next check is at 3.01 s.

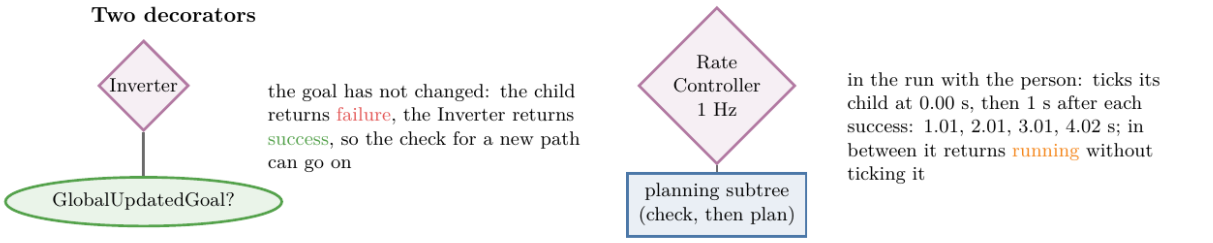
$$\text{delay} = 3.01 - 2.40$$

$$= 0.61 \text{ s}$$

Nav2's documentation gives the reason for the 1 Hz limit: without it the tree would send the planner a request at the tree's rate, 100 times a second (Nav2 docs, Detailed behavior tree walkthrough). Between two checks the controller is the part that reacts: in Nav2 it reads the local costmap every 0.05 s (`controller_frequency: 20.0`, Nav2 `nav2_params.yaml`) and steers round the person, as TEB and DWB do. Our stand-in follower does not, and at 3.01 s it is still 0.31 m from the person (notebook).

The blackboard. Nodes that must share data, such as the planner's path, which the controller then follows, do not call each other. A **blackboard** (G-3308) is a store of named entries shared by all nodes of a tree; a node writes an entry through an output port and another reads it through an input port, so nodes exchange data without knowing which node

produced it (BehaviorTree.CPP docs, "Blackboard and ports"). In Nav2's XML a port's value in braces, such as `path="{path}"`, names a blackboard entry. Figure 5, bottom, lists the entries of the default tree at 3.02 s of the blocked-door run: the planner has just failed, so `path` is empty and `compute_path_error_code` holds `NO_VALID_PATH`, which Section 7's recovery nodes read.



The blackboard

entry	value at 3.02 s	written by	read by
`goal`	(3.6, 3.5), 90°	the navigator, from the action goal	GlobalUpdatedGoal?, ComputePathToPose
`path`	empty	ComputePathToPose (emptied on failure)	IsGoalNearby?, TruncatePathLocal, FollowPath
`remaining_path`	the old path from the robot on, cut at 3.01 s	TruncatePathLocal	ValidatePath?
`compute_path_error_code`	NO_VALID_PATH	ComputePathToPose	WouldAPlannerRecoveryHelp?
`follow_path_error_code`	none	FollowPath	WouldAControllerRecoveryHelp?
`selected_controller`	FollowPath	ControllerSelector	FollowPath

Figure 5: Top: the two decorators of Nav2's default tree. The Inverter flips GlobalUpdatedGoal's failure (the goal has not changed) into success; the RateController ticks the planning part at 1 Hz, at the times of the run with the person. Bottom: the blackboard entries named by the default tree's ports, with their values at 3.02 s of our blocked-door run (notebook), the node that writes each and the nodes that read it

Sequence, fallback, decorators and the blackboard are all the parts the default tree uses, apart from three control nodes of Nav2's own. Next, the whole tree on the drive with the person.

6. Nav2's default tree, tick by tick

> **Key point:** Nav2's default tree on `main` keeps a path while the goal is unchanged, less than 4 m of path is left and the path is free, checks this once a second, and asks for a new path only when one of the three fails; the trees of the released versions (Humble, Jazzy, Kilted) remake the path at every check. With the person stepping in at 2.4 s, the tree on `main` asks for two paths, at 0.01 s and 3.02 s, and the goal is accepted at 9.77 s.

The opening's first question, whether the path is still valid and when to ask for a new one, is settled by the whole of Nav2's default tree, `navigate_to_pose_w_replanning_and_recovery.xml` (Nav2 default tree), built from the parts of Sections 3 to 5.

The two halves. The root is a `RecoveryNode` with two children (Nav2 docs, Detailed behavior tree walkthrough):

1. **navigation**, a `PipelineSequence`: five selector nodes that pick the planner, controller, goal checker, progress checker and path handler by name; then, under the 1 Hz `RateController`, the planning part; then `FollowPath`;
2. **recovery**, ticked only when navigation fails: Section 7.

A `RecoveryNode` ticks its first child; if that child fails, it ticks the second child, and if the second succeeds it ticks the first again, up to a set number of retries (Nav2 docs, Nav2-specific nodes). The root allows 6 retries; the planning part and `FollowPath` each sit in a `RecoveryNode` with 1 retry, whose second child clears the matching costmap (Section 7).

When to plan again. The planning part is a fallback whose first child is a check, `CheckIfNewPathNeeded`, a `ReactiveSequence` of four nodes that must all succeed for the old path to be kept:

- `Inverter` over `GlobalUpdatedGoal?`: the goal has not changed;
- `IsGoalNearby?`: less than 4.0 m of path is left from the robot (`proximity_threshold="4.0"`);
- `TruncatePathLocal`: writes the part of the path from the robot on to the entry `remaining_path`;
- `ValidatePath?`: for a round robot, no pose of `remaining_path` is inside an obstacle or closer to one than the robot's radius (`nav2_planner`, `is_path_valid_service.hpp`).

If any of the four fails, the fallback ticks `ComputePathToPose`. The documentation gives the reason for keeping the path near the goal: planning again close to the goal can make the robot loop, because of drift in its position estimate and errors in following the path (Nav2 docs, Detailed behavior tree walkthrough). The path in the room is short enough for that rule to apply from the start (notebook):

$$\text{path length} = 3.38 \text{ m} < 4.0 \text{ m}$$

So in the room the tree plans again only when the path is blocked. On a longer path, such as Section 9's 5.27 m path to waypoint 1, `IsGoalNearby?` fails and the tree plans again at every 1 Hz check until less than 4 m is left.

Released versions. `CheckIfNewPathNeeded` exists only in the tree on `main`. In the default trees of Nav2's released versions (Humble, Jazzy, Kilted), the `RateController` ticks `ComputePathToPose` directly, so the path is remade once a second whether or not the old one is still valid, as in the blue run of DWA and DWB (Nav2 default tree, branches `humble`, `jazzy`, `kilted`). This Note follows `main`.

On the room. Figure 6 ticks the whole tree on the drive with the person at (2.90, 2.75) from 2.4 s, RO-058's scene. Watch three frames: at 1.01 s the check succeeds and the planner is not called; at 2.40 s the person appears but only FollowPath is ticked; at 3.01 s `ValidatePath?` fails, the check fails, and `ComputePathToPose` is sent.

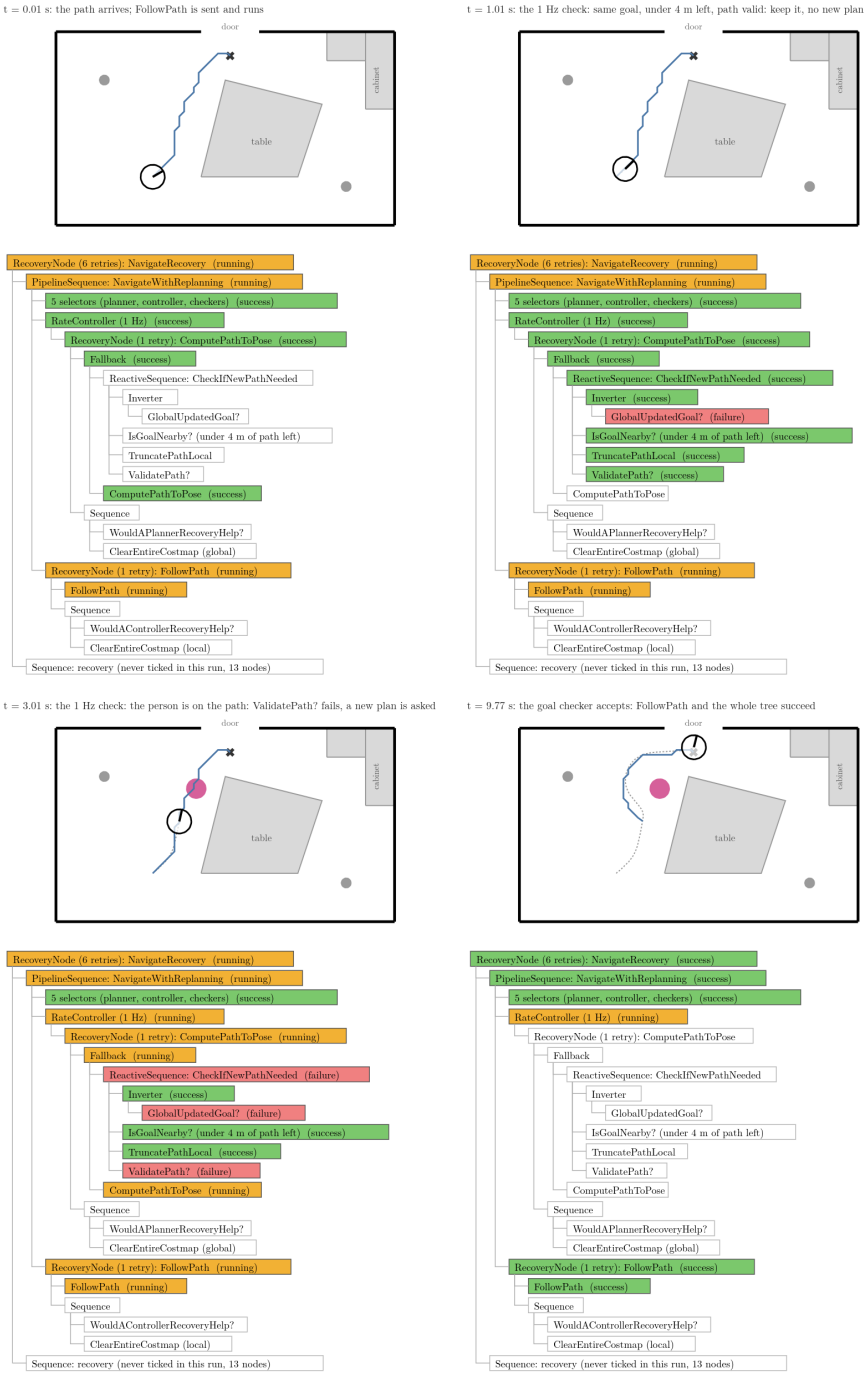


Figure 6: Animation: Nav2's default behaviour tree ticking on the drive with the person (our Python version, not Nav2 running). Top: the room; blue: the path on the blackboard; dotted: the robot's track; pink: the person from 2.4 s. Bottom: the tree; each box is coloured by what the node answered in that tick: amber running, green success, red failure, white not ticked; the recovery half is never ticked in this run and is drawn as one row. The last frame holds the end at 9.77 s, when FollowPath, and with it the whole tree, succeeds

The run's numbers (notebook):

- checks at 1.01, 2.01, 3.01, 4.02 s, . . . ; all succeed except the one at 3.01 s;
- paths asked for: two, at 0.01 s (3.38 m) and at 3.02 s (2.827 m from the robot's position then, round the person);
- goal accepted at 9.77 s, against 7.37 s in the clear room with the same tree and controller.

In this run the planner always found a path, so the recovery half was never ticked. Next, the blocked door, where the recovery half runs.

7. Recovery behaviours: when no path exists

> **Key point:** When the planner finds no path, the default tree first clears the global costmap and asks again; if that fails, it runs one recovery behaviour from a fixed round: clear both costmaps, spin 1.57 rad, wait 5 s, back up 0.30 m, and plans again after each. On the blocked door, the third recovery, the wait, lasts until the person has left, and the goal is accepted at 15.60 s; if the person stays, the tree fails after six recoveries at 13.58 s.

The opening's second question is what to try between 3.01 s and 8.0 s, when the gaps of Figure 1 leave no path.

The idea. A person whose way is blocked has a few standard responses: look again in case the obstacle was misjudged, turn to look round, wait for the way to clear, or step back. Nav2 gives the robot the same set of actions. An action that changes the robot's state or its map so that a failed step may succeed when tried again is a **recovery behaviour** (G-3309) (Nav2 docs, Navigation Servers; Zheng §6). Each of Nav2's four has its own purpose:

- **clear the costmaps:** erase the obstacle marks, for marks that come from sensor faults and no longer exist (Nav2 docs, Navigation Servers);
- **spin:** turn in place by 1.57 rad, which can bring a robot stuck in a poor heading into free space it can drive through (Nav2 docs, Navigation Servers; Zheng §6);
- **wait:** stand still for 5 s, which gives a passing problem, such as a crowd of people, time to clear (Nav2 docs, Navigate Through Poses);
- **back up:** drive 0.30 m backwards at 0.15 m/s, out of a tight spot (Nav2 docs, Navigation Servers; Zheng §6).

The spin, wait and back-up actions run in Nav2's behaviour server, `behavior_server`, the program that holds such actions and shares the local costmap between them (Nav2 docs, Navigation Servers).

First, a recovery for the failed step. When `ComputePathToPose` fails, its `RecoveryNode` (1 retry) ticks a sequence: `WouldAPlannerRecoveryHelp?`, then clear the global costmap. The condition reads `compute_path_error_code` from the blackboard and succeeds only for `NO_VALID_PATH`, `TIMEOUT` or `UNKNOWN`; a goal outside the map, for example, is not worth a recovery (`nav2_behavior_tree`, `would_a_planner_recovery_help_condition.cpp`). The same holds for `FollowPath`: its retry clears the local costmap, for the controller's errors `UNKNOWN`, `PATIENCE_EXCEEDED`, `FAILED_TO_MAKE_PROGRESS` and `NO_VALID_CONTROL`.

Then the round of four. If the step still fails, the whole navigation half fails and the root ticks the recovery half (Nav2 default tree):

1. a fallback asks whether a recovery can help, with the same two error checks;
2. a `ReactiveFallback` ticks `GoalUpdated?` first, so a new goal stops any recovery at once;
3. otherwise a `RoundRobin` ticks one of its four children: clear both costmaps, `Spin`, `Wait`, `BackUp`.

A `RoundRobin` remembers its place: after one child succeeds, the next time it is ticked it starts from the following child, and after the last it wraps round to the first (Nav2 docs, Nav2-specific nodes). Each successful recovery uses one of the root's 6 retries, and the navigation half runs again.

On the blocked door. Figure 7 ticks the whole tree on the run of Figure 1. Watch the frames at 3.02 s and 3.03 s, where one tick holds several answers of the same node, then the spin from 3.05 s, the wait from 4.78 s, and the person leaving at 8.0 s while the robot waits.

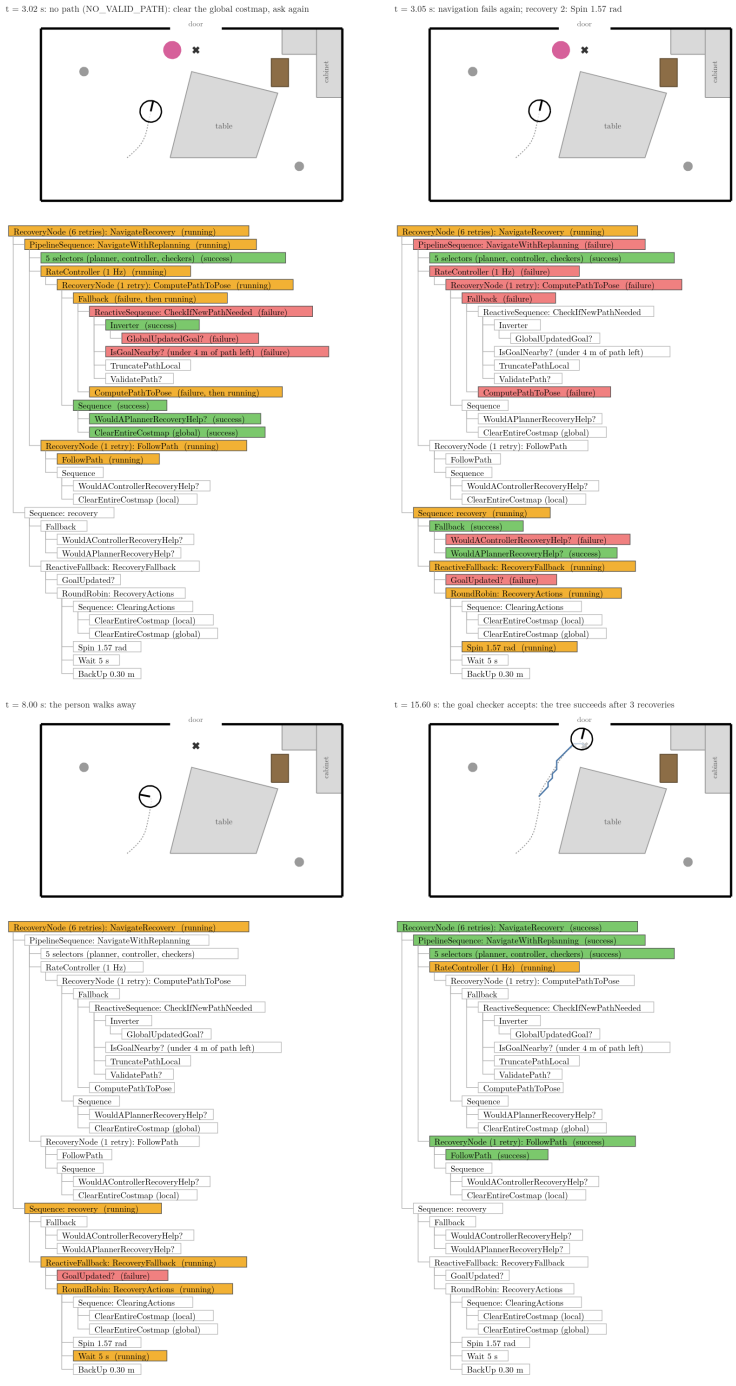


Figure 7: Animation: the blocked door ticked by Nav2's default tree (our Python version, not Nav2 running). Top: the room; pink: the person from 2.4 s to 8.0 s; brown: the trolley; blue: the path on the blackboard; dotted: the robot's track. Bottom: the whole tree; each box is coloured by what the node answered in that tick (amber running, green success, red failure, white not ticked), and a node ticked more than once in a tick lists each answer. The last frame holds the end at 15.60 s, when the goal is accepted after three recoveries

The run, tick by tick (notebook):

time (s)	what the tree does
3.01	the 1 Hz check: `ValidatePath?` fails; `ComputePathToPose` sent
3.02	`NO_VALID_PATH`: global costmap cleared; `ComputePathToPose` sent again
3.03	`NO_VALID_PATH` again: navigation fails; recovery 1: both costmaps cleared; navigation again
3.04, 3.05	`NO_VALID_PATH` twice more: navigation fails; recovery 2: `Spin` starts
4.76	spin done; navigation again
4.77, 4.78	`NO_VALID_PATH` twice: navigation fails; recovery 3: `Wait` starts
8.00	the person walks away
9.78	wait done; navigation again
9.79	a path; `FollowPath` runs
15.60	goal accepted

Clearing the costmaps did not help here, because the next sensor reading marks the person and the trolley again; the spin did not help, because the gaps stay too narrow whichever way the robot faces. Only the wait could help, because the person moves and the trolley does not.

How long a spin takes. The behaviour server commands the turn rate that would still let the robot stop exactly on the target angle at its largest angular deceleration, held between 0.4 rad/s and 1.0 rad/s (`nav2_behaviors`, `spin.cpp`; Nav2 `nav2_params.yaml`). With:

- θ, the angle to turn, 1.57 rad;
- r, the angle still to turn, in rad;
- a, the largest angular deceleration, `rotational_acc_lim`, 3.2 rad/s^2;
- ω, the commanded turn rate, in rad/s, between 0.4 and 1.0 rad/s;

$$\omega = \sqrt{2 \times 3.2 \times r}$$

$$\omega = \sqrt{2\,a\,r}$$

Why this rate: a turn that slows down at a from the rate ω turns through a further angle before it stops,

$$\text{angle to stop} = \omega^2/(2a)$$

and setting that angle equal to r gives the rate above, the fastest rate from which the turn can still stop on the target. The time of the whole spin comes in three parts, with:

- $\omega_{\max}$, the largest turn rate, 1.0 rad/s;
- $\omega_{\min}$, the smallest turn rate, 0.4 rad/s;
- r_{full}, the angle still to turn when the rate starts to fall below 1.0 rad/s, in rad;
- $r_{\min}$, the angle still to turn when the rate reaches 0.4 rad/s, in rad;
- t_{full}, the time at 1.0 rad/s, in s;
- t_{slow}, the time while the rate falls from 1.0 to 0.4 rad/s, in s;
- t_{last}, the time at 0.4 rad/s, in s;
- t, the time of the whole spin, in s.

$$r_{\text{full}} = 1.0^2/(2 \times 3.2)$$

$$= 0.1563 \text{ rad}$$

$$t_{\text{full}} = (1.57 - 0.1563)/1.0$$

$$= 1.414 \text{ s}$$

$$r_{\min} = 0.4^2/(2 \times 3.2)$$

$$= 0.0250 \text{ rad}$$

$$t_{\text{slow}} = 2(\sqrt{0.1563} - \sqrt{0.0250})/\sqrt{6.4}$$

$$= 0.188 \text{ s}$$

$$t_{\text{last}} = 0.0250/0.4$$

$$= 0.0625 \text{ s}$$

$$t = 1.414 + 0.188 + 0.0625$$

$$= 1.66 \text{ s}$$

The same steps in symbols:

$$r_{\text{full}} = \omega_{\max}^2/(2a)$$

$$t_{\text{full}} = (\theta - r_{\text{full}})/\omega_{\max}$$

$$t_{\text{slow}} = 2(\sqrt{r_{\text{full}}} - \sqrt{r_{\min}})/\sqrt{2a}$$

$$t_{\text{last}} = r_{\min}/\omega_{\min}$$

In the run the spin takes 1.71 s, from 3.05 s to 4.76 s, and turns 1.60 rad: the server changes the command only every 0.1 s (`cycle_frequency: 10.0`), so it stops at the first check after the target angle (notebook). The wait takes exactly 5.00 s, from 4.78 s to 9.78 s.

When the person stays. With the person never leaving, the round goes on: clear both costmaps at 3.03 s, spin from 3.05 s, wait from 4.78 s, back up 0.30 m from 9.80 s to 11.81 s, clear both costmaps at 11.83 s, spin from 11.85 s. After that sixth recovery the planner still finds no path, the root has used its 6 retries, and the tree fails at 13.58 s (notebook). The navigation action then ends as aborted, which is the result the goal's sender receives (`nav2_behavior_tree`, `bt_action_server_impl.hpp`; `nav2_ros_common`, `simple_action_server.hpp`). From the first recovery, at 3.03 s, to the failure, the time is the recoveries plus the ticks spent asking the planner between them:

$$\text{whole span} = 13.58 - 3.03$$

$$= 10.55 \text{ s}$$

$$\text{recoveries} = 1.71 + 5.00 + 2.01 + 1.71$$

$$= 10.43 \text{ s}$$

$$\text{ticks between} = 10.55 - 10.43$$

$$= 0.12 \text{ s}$$

The 0.12 s are 12 ticks of 10 ms, in which the planner was asked again and answered that no path exists. The tree now finds a path again whenever one appears. What it has not shown is how it decides that the goal is reached, or that the robot is stuck while a path exists. Next, the two checks that decide that.

8. The goal checker and the progress checker

> **Key point:** The goal checker accepts the goal once the robot is within 0.25 m of the goal position and then within 0.25 rad of its heading, and does not check the position again while the robot turns. The progress checker fails `FollowPath` when the robot has not moved 0.5 m from its last checkpoint in 10 s.

The opening's third question: when does the goal count as reached, and when does the robot count as stuck?

The idea. A robot never stops exactly on a pose: localisation and control leave errors of centimetres. A rule must say how close is close enough. And a robot whose wheels turn while it stands still, held by a cable or a doorstep, looks to the planner like a robot on its way; another rule must notice that nothing is changing.

The standard terms. Both checks run inside Nav2's controller server, on every control cycle (`nav2_controller`, `simple_goal_checker.cpp` and `simple_progress_checker.cpp`; Nav2 `nav2_params.yaml`):

- a **goal checker** (G-3310) is the test the controller runs to decide that the robot has reached its goal; Nav2's default requires less than `path_length_tolerance`, 1.0 m, of path left, the position within `xy_goal_tolerance`, 0.25 m, and then the heading within `yaw_goal_tolerance`, 0.25 rad, and with `stateful: true` it keeps the position as reached while the robot turns on the spot;
- a **progress checker** (G-3311) is the test that decides the robot is stuck; Nav2's default stores a checkpoint, moves it to the robot whenever the robot is more than `required_movement_radius`, 0.5 m, from it, and fails when `movement_time_allowance`, 10 s, passes without that.

Figure 8 shows both on our runs. Top, the end of the blocked-door run: look at the moment the blue distance crosses 0.25 m, then at the orange heading error falling to 0.25 rad. Bottom, a run in which a wheel is caught at 2.0 s: look at the flat line after the checkpoint at 1.57 s and the red failures 10 s later.

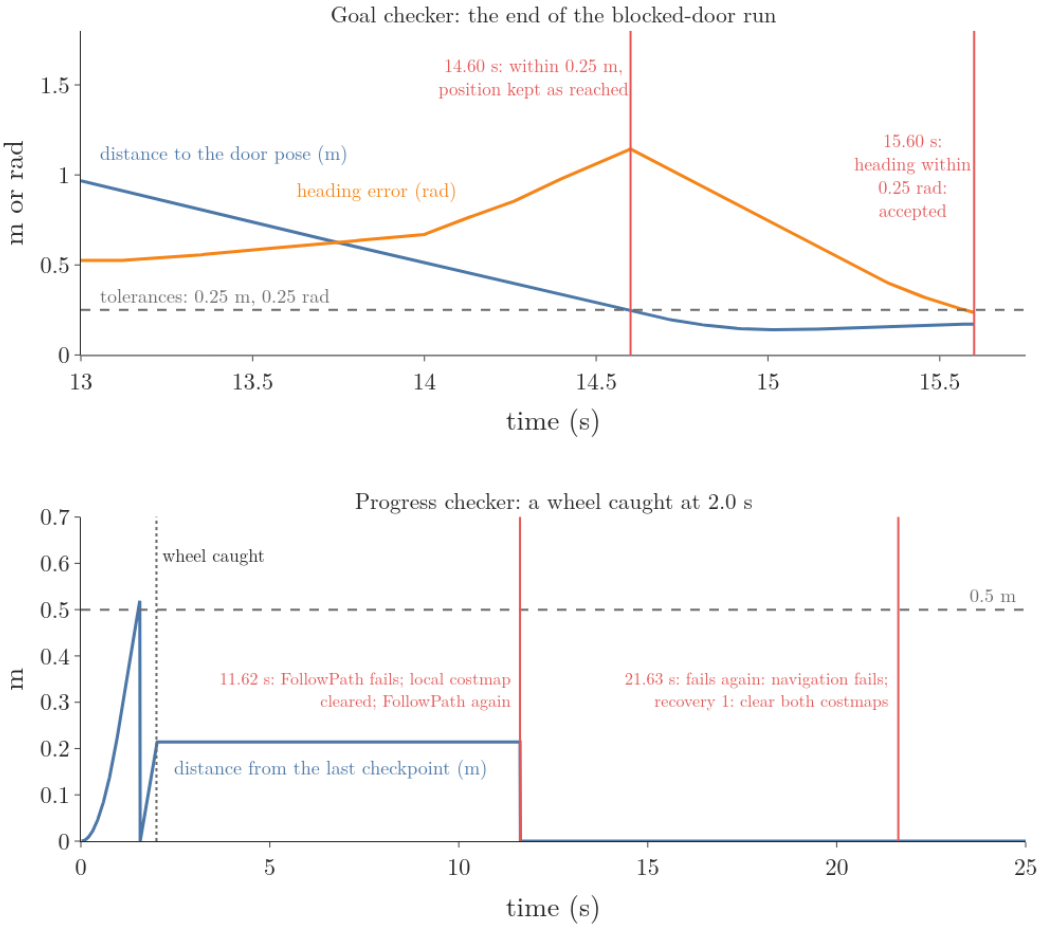


Figure 8: Nav2's goal checker and progress checker in our runs (our Python version, not Nav2 running). Top: the last 2.6 s of the blocked-door run; blue: the distance from the robot to the door pose (m); orange: the heading error (rad); grey dashed: the two tolerances, 0.25 m and 0.25 rad. Bottom: a run in which a wheel is caught at 2.0 s; blue: the distance from the progress checker's last checkpoint (m); grey dashed: 0.5 m; red: the two progress failures, at 11.62 s and 21.63 s

The goal checker on the blocked door (notebook):

$$\text{distance at } 14.60 \text{ s} = 0.244 \text{ m}$$

$$\leq 0.25 \text{ m}$$

$$\text{heading error at } 15.60 \text{ s} = 0.236 \text{ rad}$$

$$\leq 0.25 \text{ rad}$$

At 14.60 s the heading is still 1.144 rad off, so the robot turns on the spot. It is still slowing down from its forward speed while it turns, so the distance does not stay at 0.244 m: it falls to 0.140 m and is 0.171 m at 15.60 s. The stateful checker does not test it again; the goal is accepted at 15.60 s, 0.171 m and 0.236 rad, or 13.54°, from the door pose. Our version leaves out the path-length test: at 14.60 s the path left is 0.271 m, under 1.0 m, so the test would not change the result (notebook).

The progress checker on the stuck run (notebook): the checkpoint moved to the robot at 1.57 s, the robot moved 0.214 m further before the wheel caught, and did not move after that:

$$\text{time without progress} = 11.62 - 1.57$$

$$= 10.05 \text{ s} > 10 \text{ s}$$

$$\text{distance moved} = 0.214 \text{ m} < 0.5 \text{ m}$$

At 11.62 s `FollowPath` fails with `FAILED_TO_MAKE_PROGRESS`. That error passes `WouldAControllerRecoveryHelp?`, so the tree clears the local costmap and starts `FollowPath` again, with a new checkpoint. At 21.63 s it fails again, the navigation half fails, and the round of Section 7 starts with clearing both costmaps. The progress checker turns "stuck" into a failure the recoveries can act on.

The tree can now reach one goal, recover, and tell success from being stuck. Many tasks need several goals in a row. Next, how Nav2 sends them.

9. Waypoint following

> **Key point:** Nav2's waypoint follower sends a list of poses one at a time, each as its own navigation goal; after each pose reached it runs a task, by default a 200 ms pause, and with `stop_on_failure: false` it records a failed pose as missed and goes on to the next. In the room, waypoint 2, which the person and the trolley shut off, is missed after six recoveries and waypoint 3 is reached at 34.41 s.

The opening's robot had one goal. A patrol, a delivery round or an inspection has a list, and the person at the door may block one pose of the list while the others stay reachable.

The idea. A delivery driver with a list of addresses drives to each in turn, leaves the parcel, and drives on; if one street is closed, the driver notes the address as missed and goes on to the next one instead of ending the round.

The standard term. Waypoint following (G-3312) is driving a robot through a list of poses in order, sending each as a separate navigation goal and, at each pose reached, running a task such as a pause or a photograph; the program that does it in Nav2 is `waypoint_follower` (Nav2 docs, Navigation Servers). Its rules, from its

code and Nav2's bring-up file (`nav2_waypoint_follower`, `waypoint_follower.cpp` and `wait_at_waypoint.cpp`; Nav2 `nav2_params.yaml`):

1. send waypoint i as a `NavigateToPose` goal and wait for its result, checking 20 times a second (`loop_rate: 20`);
2. if it succeeds, run the task executor at the waypoint: `WaitAtWaypoint`, a pause of 200 ms (`waypoint_pause_duration: 200`);
3. if it fails, add it to the list of missed waypoints; with `stop_on_failure: false` go on, otherwise stop the whole list;
4. go on to waypoint i + 1; at the end, report the missed waypoints.

Nav2 also offers `NavigateThroughPoses`, a single navigation goal that plans one path from the robot through all the poses to the last (Nav2 docs, Navigate Through Poses); the robot then does not stop at each pose, and no task runs there.

On the room. Three waypoints, with the person in the gap and the trolley in place from the start: waypoint 1 at (6.0, 2.0) facing 90°, right of the table; waypoint 2 at (4.6, 3.5) facing 180°, above the table, which both blocked gaps shut off; waypoint 3 at (1.5, 2.0) facing 180°, left of the table. Figure 9 plays the round. Watch waypoint 2 turn red at 21.24 s while the robot, which never left the area near waypoint 1, drives on to waypoint 3.

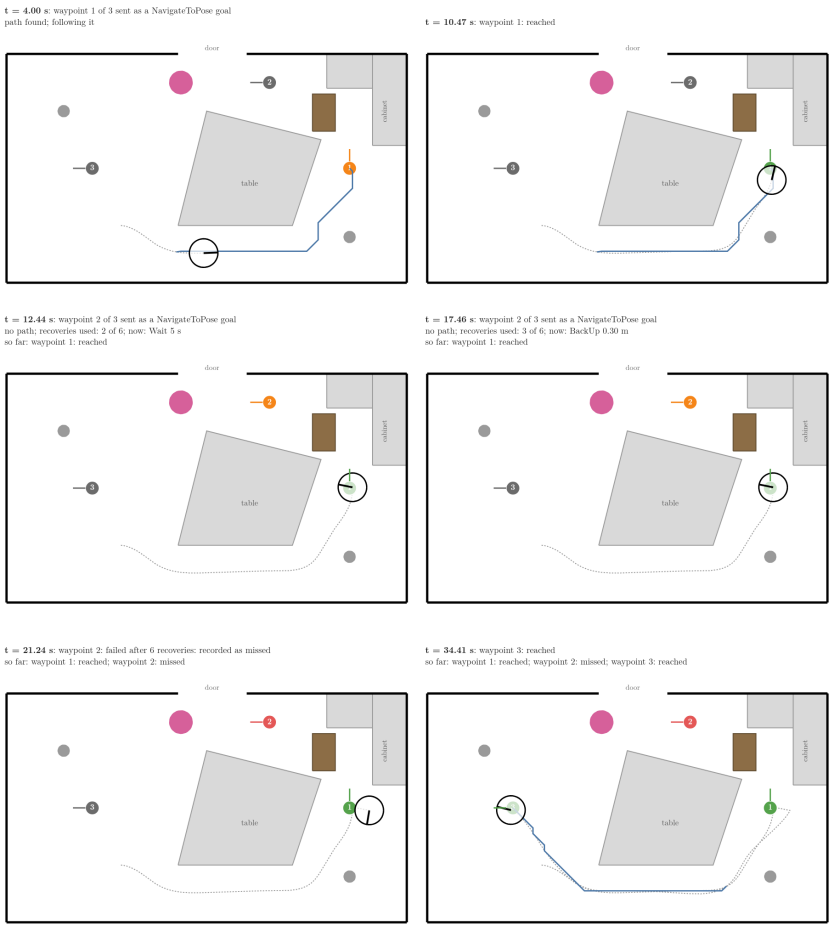


Figure 9: Animation: waypoint following in the room (our Python version of Nav2's waypoint follower and default tree, not Nav2 running). Numbered circles: the three waypoints with their headings; grey: not yet sent; orange: being driven to; green: reached; red: missed. Pink: the person; brown: the trolley; blue: the path on the blackboard; dotted: the robot's track. The text above names the waypoint, the recovery running and the results so far. The last frame holds the end at 34.41 s: waypoints 1 and 3 reached, waypoint 2 missed

The round (notebook):

waypoint	sent (s)	ended (s)	result
1, (6.0, 2.0), 90°	0.00	10.47	reached
2, (4.6, 3.5), 180°	10.67	21.24	failed: missed
3, (1.5, 2.0), 180°	21.24	34.41	reached

Waypoint 2 starts 0.20 s after waypoint 1 ends, the pause of `WaitAtWaypoint`. For waypoint 2 the tree runs the same six recoveries as Section 7's run with the person staying: clear, spin, wait, back up, clear, spin, from 10.69 s to 21.22 s. The path to waypoint 1 is 5.27 m long,

more than 4 m, so the tree plans it again at 1.02, 2.03 and 3.04 s, as Section 6 said; from 4.05 s less than 4 m is left and the path is kept.

The robot can now run a list of goals and recover from blocked paths with Nav2's tree. Cars on public roads face the same choice of what to do next, with different tasks. Next, how two driving stacks make that choice.

10. How cars switch behaviours: Apollo and Autoware

> **Key point:** Apollo's planner chooses one scenario at a time from a list in priority order and runs it as a fixed chain of stages, each naming the next, a state machine; Autoware's behaviour path planner runs scene modules in slots, and a module that asks to run joins a candidate list and, once approved, an approved list run in series.

The opening's robot needed to decide when to replan, recover and stop. A car must decide when to follow its lane, stop at a sign, give way, change lanes or pull over, every 0.1 s, and two open-source driving stacks make that choice in two different ways.

10.1 Apollo: scenarios and stages

The idea. A driver approaching a stop sign runs through fixed steps: slow to stop at the line, wait until the cars that arrived first have gone, edge forward to see, then cross. Apollo writes each such situation as a fixed list of steps.

The standard terms. A **scenario** (G-3313) in Apollo's planning module is one driving situation with its own planning steps, such as following the lane, a stop sign or pulling over; a scenario manager picks one per planning cycle. A **stage** (G-3314) is one step of a scenario, with its own rule for when it ends and which stage comes next. Their rules, read from Apollo's code (Apollo, `modules/planning`):

- planning runs every 0.1 s (`planning_loop_rate`, 10);
- the scenario manager checks the scenarios in the configured order, from `EMERGENCY_PULL_OVER` to `LANE_FOLLOW`, 12 in all (`planner_config.pb.txt`); it switches to the first one that applies, but keeps the current one while it is running unless one earlier in the list applies (`scenario_manager.cc`);
- when a stage ends it names the next stage; when the last ends, the scenario is done and the manager chooses again (`scenario.cc`).

The stages of the stop-sign scenario, `STOP_SIGN_UNPROTECTED`, and the rules that end them (Apollo, folder `stop_sign_unprotected`, `scenario_conf.pb.txt`):

1. **PRE_STOP**, entered when the car's front is 4.0 m or less before the stop line: ends when the car has stopped, at 0.2 m/s or less, at most 2.0 m before the line; meanwhile it puts every vehicle within 5.0 m of its own stop line on a crossing lane on a watch list;
2. **STOP**: ends after at least 1.0 s, once every watched vehicle is more than 10.0 m from its stop line, or after 8.0 s with at most one left;
3. **CREEP**: edges forward; ends when the front is 2.0 m past the line, or after 10.0 s, and no moving obstacle can reach the car's path within 6.0 s, for 5 cycles in a row;

4. **INTERSECTION_CRUISE**: ends when the car's rear is 20.0 m past the line, when the map has no junction outline; the scenario is then done.

On a crossing. The room has no stop sign, so a new example follows: a car on a straight road with a stop sign, a crossing car already stopped at its own line, our own one-dimensional model (`images/apollokit.py`) with Apollo's stage rules and our own speeds. Figure 10 plays it. Watch the stage panel: PRE_STOP from 5.5 s, STOP from 7.4 s, and the change to CREEP only at 13.2 s, after the crossing car has left.

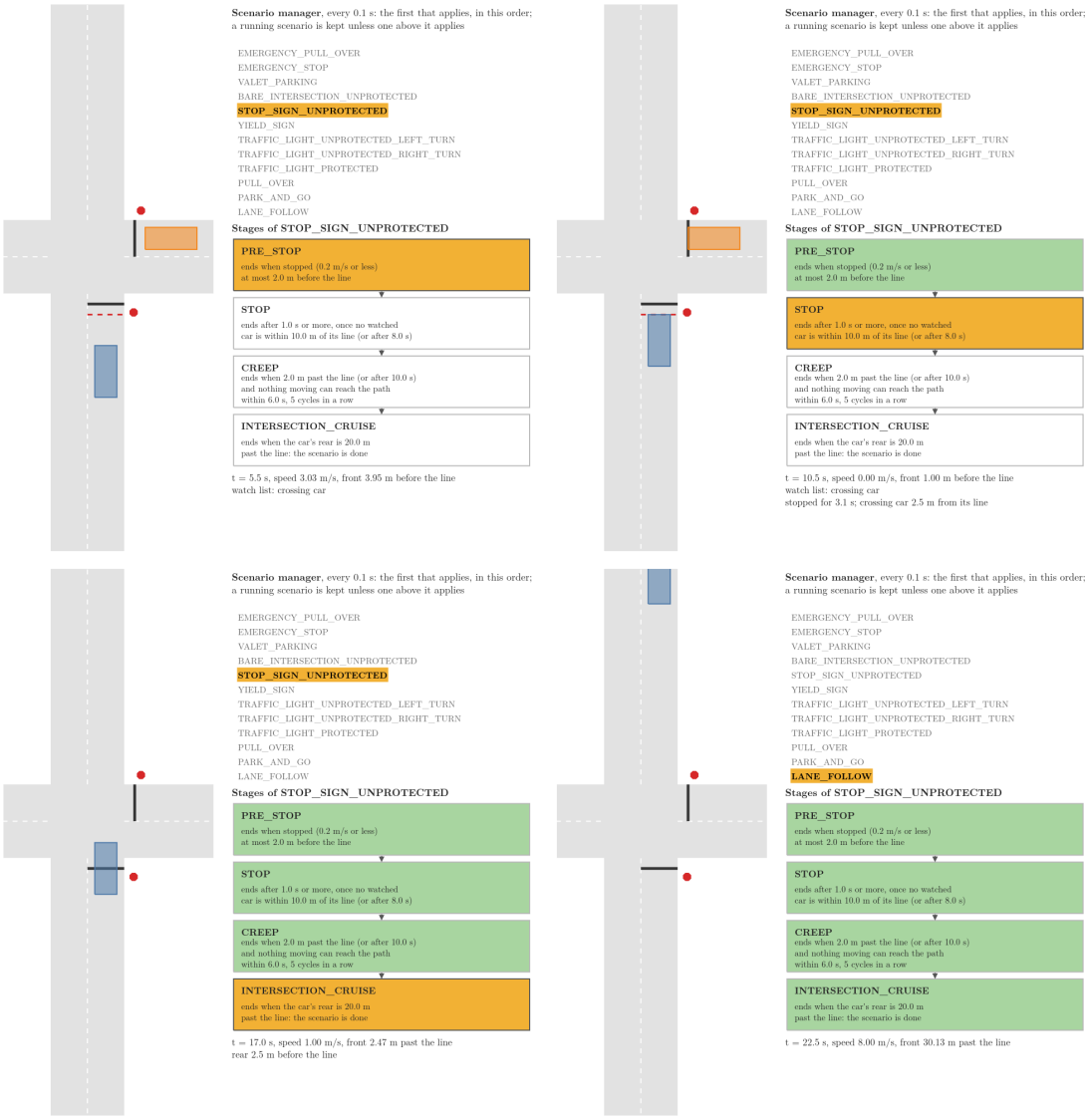


Figure 10: Animation: Apollo's stop-sign scenario on our one-dimensional model (images/apollokit.py: the scenario and stage rules copied from Apollo's code, the speeds ours; not Apollo running). Left: the crossing from above; blue: our car, Apollo's Lincoln MKZ, 4.933 m long, going up at 8 m/s and braking at 1.5 m/s^2; orange: a car that stopped first at its own stop line and leaves at 9.5 s; dark lines: the stop lines; red dashed: the stop fence 1.0 m before our line; grey dashed: the creep fence 4.0 m past it. Right: the scenarios in the manager's order, the one chosen in amber; the four stages, the running one in amber and those finished in green; below, the numbers the current stage's rule reads in that cycle. The last frame holds 22.5 s, back in LANE_FOLLOW

The stage changes (notebook): `LANE_FOLLOW` until 5.5 s; PRE_STOP from 5.5 s; STOP from 7.4 s, the car stopped 1.00 m before the line; CREEP from 13.2 s; INTERSECTION_CRUISE from 17.0 s; back to `LANE_FOLLOW` at 21.9 s. Why STOP lasts until 13.2 s: the crossing car's centre starts 3.47 m from its line (front 1.0 m before it, half its 4.933 m length behind that)

and must be 10.0 m past it, so it travels 13.47 m from rest at our 2.0 m/s^2:

$$\text{time} = \sqrt{2 \times 13.47/2.0}$$

$$= 3.67 \text{ s}$$

$$\text{clear at} = 9.5 + 3.67$$

$$= 13.17 \text{ s}$$

The stage ends at the next 0.1 s cycle, 13.2 s. Each stage names the one after it, so the stages form a finite state machine of Section 2's kind, with four states in a line; the choice between scenarios, the first in a fixed order that applies, is the same rule as a fallback's. Apollo's planning in full comes in Apollo planning and control in depth (RO-236).

10.2 Autoware: scene modules in slots

The idea. Some driving tasks must run together: a car changing lanes may meet a parked car in the new lane and must steer round it while the lane change goes on. A single active state cannot express that.

The standard term. A **scene module** (G-3315) in Autoware's behaviour path planner is a plug-in for one driving task, such as a lane change, avoiding a parked car or pulling into a goal, that asks to run when its situation arises and changes the path it is given (Autoware, manager design document). Its rules, from that design document and the code on `main` (Autoware, `scene_module_manager.param.yaml`, `planner_manager.cpp`, `scene_module_interface.hpp`):

- the planner runs 10 times a second (`planning_hz: 10.0`, `behavior_path_planner.param.yaml`) and passes the lane's centre line through four slots in turn, each slot taking the path from the one before;
- within a slot, a module listed earlier has priority;
- a module that asks to run becomes a candidate if the approved modules allow it; candidates run side by side on the approved path, waiting for approval;
- an approved module moves to the approved stack, where modules run in series, each taking the path of the one before;
- each module is in one of five states, `IDLE`, `RUNNING`, `WAITING_APPROVAL`, `SUCCESS` and `FAILURE`; a module that fails is removed together with every module approved after it.

Figure 11 shows the four slots of the default configuration, a module's states, and the design document's example of avoiding a parked car during a lane change.

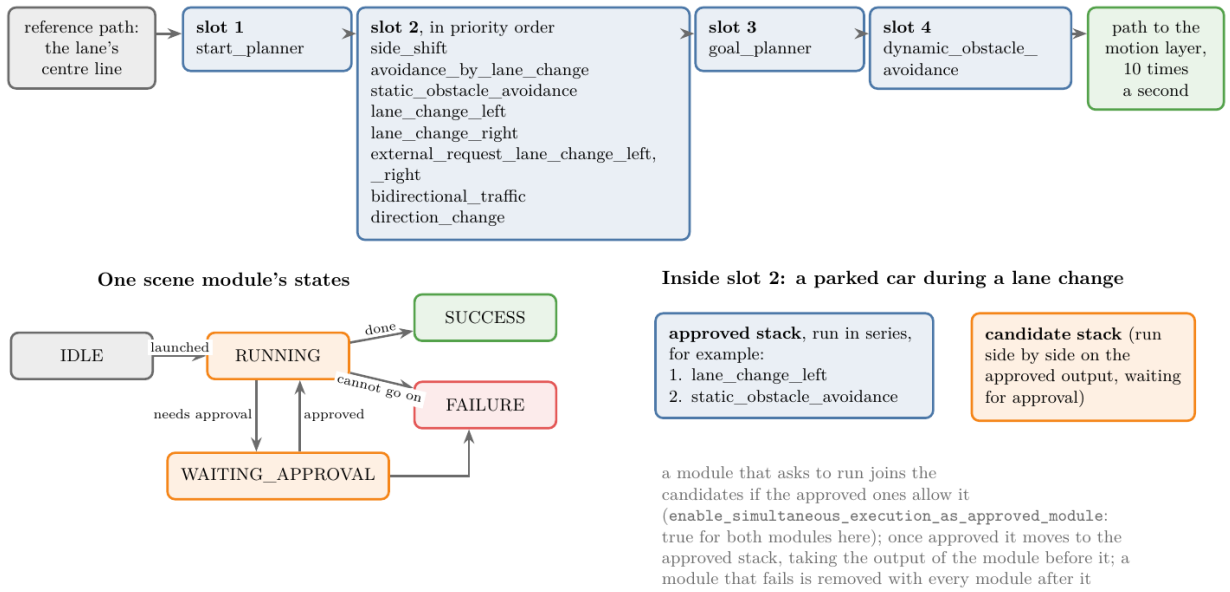


Figure 11: Autoware's behaviour path planner (main, read 2026-10-12). Top: the four slots of scene_module_manager.param.yaml, with their modules in priority order; each slot takes the path from the one before and the last passes it to the motion layer 10 times a second. Bottom left: one scene module's states, from updateState(). Bottom right: inside slot 2, the approved stack, run in series, with a lane change and the avoidance of a parked car both approved, and the candidate stack

Autoware's manager is neither a tree nor a single chain: several modules can be active at once, in an order set by the slots. Autoware's planning in full comes in Autoware planning and control in depth (RO-234).

10.3 Three ways compared

	Nav2	Apollo	Autoware
structure	behaviour tree in an XML file	scenarios in priority order, each a chain of stages	scene modules in four slots
rate	tick every 10 ms	every 0.1 s	every 0.1 s
what runs at once	one branch of the tree	one scenario, one stage	several approved modules, in series
what comes next	parents decide from the answers	each stage names the next	modules ask to run; the manager admits them
when a step fails	recoveries, then the action aborted after 6	a stage error ends the cycle with a planning error	the failed module and those approved after it removed

Each structure matches what its stack must do. Nav2 adds its recoveries as one branch of the tree without changing the tasks (Nav2 docs, Detailed behavior tree walkthrough). Apollo's stop sign always runs its four stages in the same order, so each stage can name the next. Autoware's slots and stacks exist so that modules such as a lane change and an avoidance can run at the same time (Autoware, manager design document).

11. Summary

Question	Answer	In the room
what comes next	a tick from the root; nodes answer success, failure or running	978 ticks for the 9.77 s drive
combining steps	sequence “and”, fallback “or”; condition first: check before acting	the path checked, the planner called only on failure
how often to check	a rate controller decorator, 1 Hz	the person noticed 0.61 s after stepping in
passing data	the blackboard	`path` emptied and `NO_VALID_PATH` written at 3.02 s
no path	clear the global costmap; then clear both, spin, wait, back up in turn	the wait lasts until the person has left; goal at 15.60 s
no path ever appears	6 recoveries, then failure	aborted at 13.58 s
goal reached	0.25 m, then 0.25 rad, stateful	14.60 s, then 15.60 s
stuck	0.5 m in 10 s	failure at 11.62 s
several goals	one goal each, missed ones recorded	waypoint 2 missed, waypoint 3 at 34.41 s

- On the blocked door, our Python version of Nav2’s default tree detected the blocked path at 3.01 s, cleared the costmaps, spun 1.57 rad in 1.71 s and waited 5 s, so a path existed again at 9.79 s and the goal was accepted at 15.60 s, because the wait was the only recovery long enough for a moving person to leave.
- A **finite state machine** (G-3299) needs 12 transitions and a counter for these recoveries and 7 more to add docking, because each state must name its own next state.
- A **behaviour tree** (G-3300) lets each task answer only how it went, its **return status** (G-3302), to a **tick** (G-3301) from its parent, so a task can be added or moved without editing the others.
- **Action nodes** (G-3303) do things and may run for many ticks; **condition nodes** (G-3304) only check, so they answer at once.
- A **sequence node** (G-3305) runs steps that must all succeed; a **fallback node** (G-3306) tries options in order, so a condition first in a fallback skips a task whose goal already holds.
- Nav2 ticks its planning branch again on every tick through a pipeline sequence, because a plain sequence would never check again a path it had already accepted.
- A **decorator node** (G-3307) such as the 1 Hz rate controller limits how often the path is checked and the planner asked, so the tree detects a blocked path up to 1 s late and the controller reacts in between.
- The **blackboard** (G-3308) carries the path and the error codes between nodes, so the

recoveries can test whether the error is one a recovery can fix.

- A **recovery behaviour** (G-3309) changes the map or the robot's pose so that planning can be tried again; Nav2 takes them in a round and aborts after 6, so a goal that cannot be reached ends as aborted, here 10.55 s after the first recovery.
- The **goal checker** (G-3310) accepts 0.25 m and 0.25 rad, and keeps the position as reached while the robot turns; the **progress checker** (G-3311) turns a robot that does not move 0.5 m in 10 s into a failure the recoveries can act on.
- **Waypoint following** (G-3312) sends one goal per pose and records the missed ones, so one blocked pose does not end the round.
- Apollo runs one **scenario** (G-3313) at a time as a chain of **stages** (G-3314), a state machine; Autoware runs **scene modules** (G-3315) in slots, several at once, because a car's tasks can overlap.

The robot can now choose between planning, driving, recovering and aborting, for one goal or a list. One limitation remains: how the servers round the tree, from the map and the robot's position estimate to the commands at the wheels, start up and work together on one goal; Nav2 start to finish (RO-080) follows one goal through all of them. After that, choosing the task itself from a request, such as which object to fetch and in what order, is task and motion planning (RO-081).

12. Sources

Built from

- Petter Ögren, "5 minute Behavior Tree tutorial", YouTube, https://www.youtube.com/watch?v=KeShI the teaching order; the problem of choosing what to do next and why every transition of a finite state machine must be known; the tick and the three answers; fallback as "or" and sequence as "and"; one tree run step by step, with running passed up to the root
- Nikolaus Correll, "Lecture III.4 - Arm Kinematics", YouTube, https://www.youtube.com/watch?v=6yF 0:06–4:24 only: checking a condition before running a sequence, the tick rate, the blackboard, and the repeat and invert decorators
- Robotics Back-End, "ROS2 Nav2 - Navigation Stack in 1 Hour [Crash Course]", YouTube, https://www.youtube.com/watch?v=idQb2pB-h2Q, 51:11–52:15 and 54:26–58:44: a goal that cannot be reached ending as aborted after recoveries, and waypoint following against navigating through poses (Robotics Back-End, Nav2 crash course)
- Colledanchise, M. and Ögren, P. (2018). *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, arXiv 1709.00084: §1.2 (finite state machines and one-way transitions), §1.3 and Table 1.1 (tick, sequence, fallback, decorator, action and condition nodes), §2.1 (finite state machines), §3.1 (explicit success conditions) (Colledanchise and Ögren)
- BehaviorTree.CPP documentation, https://www.behaviortree.dev (version 4): "Introduction to BT basics", "Blackboard and ports", "Sequences" (BehaviorTree.CPP docs)
- BehaviorTree.CPP, GitHub, https://github.com/BehaviorTree/BehaviorTree.CPP, folder `src`: `control_node.cpp`, `tree_node.cpp`, and in `controls`: `sequence_node.cpp`,

fallback_node.cpp, reactive_sequence.cpp, reactive_fallback.cpp
- Nav2 documentation, https://docs.nav2.org (source ros-navigation/docs.nav2.org, branch rolling, read 2026-10-12): "Behavior Trees", "Navigation Servers" and "Environmental Representation" in Navigation Concepts, "Detailed Behavior Tree Walkthrough", "Nav2 Specific Nodes", "Navigate Through Poses" (Nav2 docs)
- navigation2, GitHub, https://github.com/ros-navigation/navigation2 (branch main, read 2026-10-12), by package:
  - nav2_bt_navigator, folder behavior_trees: navigate_to_pose_w_replanning_and_recover
    and subtrees/backup_with_reset.xml (Nav2 default tree), and the same tree file on the branches humble, jazzy and kilted, read 2026-10-12, for the released versions;
  - nav2_behavior_tree: bt_action_server_impl.hpp (a failed tree aborts the goal), and in the folder plugins: pipeline_sequence.cpp, recovery_node.cpp, round_robin_node.cpp, rate_controller.cpp, is_goal_nearby_condition.cpp, goal_updated_condition.cpp, would_a_planner_recovery_help_condition.cpp, would_a_controller_recovery_help_condition.cpp, compute_path_to_pose_action.cpp, follow_path_action.cpp, validate_path_action.cpp, truncate_path_local_action.cpp; the headers spin_action.hpp and back_up_action.hpp (time allowance 10 s);
  - nav2_ros_common: simple_action_server.hpp;
  - nav2_planner: is_path_valid_service.hpp (the test behind ValidatePath);
  - nav2_behaviors: spin.cpp and drive_on_heading.hpp (spin speed, back-up collision check);
  - nav2_controller: simple_goal_checker.cpp and simple_progress_checker.cpp;
  - nav2_waypoint_follower: waypoint_follower.cpp and wait_at_waypoint.cpp;
  - nav2_bringup: params/nav2_params.yaml, for bt_loop_duration, the behaviour server, the goal and progress checkers and the waypoint follower (Nav2 nav2_params.yaml)
- Zheng, K. (2017). "ROS Navigation Tuning Guide". arXiv 1706.09068, §6 recovery behaviours (Zheng)
- Apollo, GitHub, https://github.com/ApolloAuto/apollo (branch master, read 2026-10-12), folder modules/planning (Apollo):
  - planners/public_road: scenario_manager.cc, public_road_planner.cc, conf/planner_config.pb.txt;
  - planning_interface_base/scenario_base: scenario.cc, stage.cc, base_stage_creep.cc, base_stage_cruise.cc;
  - scenarios/stop_sign_unprotected: stop_sign_unprotected_scenario.cc, stage_pre_stop.cc, stage_stop.cc, stage_creep.cc, stage_intersection_cruise.cc, conf/scenario_conf.pb.txt, proto/stop_sign_unprotected.proto;
  - traffic_rules/stop_sign/conf/default_conf.pb.txt; planning_base/gflags/planning_
    and modules/common/data/vehicle_param.pb.txt
- Autoware, GitHub, https://github.com/autowarefoundation/autoware_universe (branch main, read 2026-10-12), folder planning/behavior_path_planner (Autoware):
  - autoware_behavior_path_planner: docs/behavior_path_planner_manager_design.md, config/scene_module_manager.param.yaml, config/behavior_path_planner.param.yaml,

`src/planner_manager.cpp`, `planner_manager.hpp`;
  – `autoware_behavior_path_planner_common`: `scene_module_interface.hpp`

Other references

- Macenski, S., Martín, F., White, R. and Ginés Clavero, J. (2020). "The Marathon 2: A Navigation System". *IEEE/RSJ IROS*, arXiv 2003.00368: Nav2's use of behaviour trees to order navigation tasks
- Robohub, "Introduction to Behavior Trees", https://robohub.org/introduction-to-behavior-trees/: a robot's tree built step by step and compared with finite state machines

13. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Finite state machine (FSM) (G-3299)	A model with a fixed set of states, one of them active at a time, and transitions between them, each triggered by an event, such as Plan, Drive and Spin for a robot; it fixes what comes after each task, but every state must name its own next states, so adding a task means editing many states.
Behaviour tree (BT) (G-3300)	A tree whose leaves are tasks and checks and whose inner nodes decide, from the leaves' answers, which leaf runs next, such as Nav2's tree that plans, follows the path and recovers; no task names another, so tasks can be added or moved without editing the rest.
Tick (G-3301)	The signal a parent node sends to a child to run one step, sent from a behaviour tree's root at a fixed rate, such as every 10 ms in Nav2; a node runs only while it receives ticks.
Return status (behaviour tree) (G-3302)	A node's answer to a tick: success when its task is done or its check holds, failure when it cannot be done or the check does not hold, running when it needs more time; the parent decides from it what to tick next.
Action node (behaviour tree) (G-3303)	A leaf of a behaviour tree that does something, such as asking the planner for a path or turning in place, and answers success, failure or running; it may run for many ticks.

Term	Meaning
Condition node (G-3304)	A leaf of a behaviour tree that checks something, such as whether the path is still free, and answers success or failure at once, never running; it lets the tree skip or choose tasks.
Sequence node (G-3305)	A control node that ticks its children from the left and answers failure at the first child that fails, running while a child runs and success when all have succeeded, such as plan and then follow the path; it runs steps that must all happen in order.
Fallback node (selector) (G-3306)	A control node that ticks its children from the left and answers success at the first child that succeeds, running while a child runs and failure when all have failed, such as keep the path if it is valid, else plan a new one; it tries options in order of preference.
Decorator node (G-3307)	A control node with exactly one child that changes when the child is ticked or what its answer becomes, such as an inverter that swaps success and failure or a rate controller that lets a tick through once a second.
Blackboard (G-3308)	A store of named entries shared by all the nodes of a behaviour tree, such as the goal, the path and the planner's error code; nodes write and read entries through their ports, so they exchange data without knowing one another.
Recovery behaviour (G-3309)	An action that changes the robot's pose or its map so that a failed step, such as planning a path, may succeed when tried again; Nav2's are clearing the costmaps, spinning, waiting and backing up, taken in turn.
Goal checker (G-3310)	The test a controller runs to decide that the robot has reached its goal, such as Nav2's default: the position within 0.25 m, then the heading within 0.25 rad, with the position kept as reached while the robot turns; it ends the path-following action with success.

Term	Meaning
Progress checker (G-3311)	The test a controller runs to decide that the robot is stuck, such as Nav2's default: a failure when the robot has not moved 0.5 m from its last checkpoint in 10 s; it turns a robot that does not move into an error the recovery behaviours can act on.
Waypoint following (G-3312)	Driving a robot through a list of poses in order, sending each as a separate navigation goal and running a task, such as a pause, at each one reached; a failed pose can be recorded as missed so the rest of the list still runs.
Scenario (Apollo planning) (G-3313)	One driving situation in Apollo's planning module with its own planning steps, such as following the lane, a stop sign or pulling over; a scenario manager picks one per 0.1 s cycle, the first in a fixed order that applies.
Stage (Apollo planning) (G-3314)	One step of an Apollo scenario, such as stopping at the line or creeping forward at a stop sign, with its own rule for when it ends and which stage comes next; the stages of a scenario form a state machine.
Scene module (Autoware) (G-3315)	A plug-in of Autoware's behaviour path planner for one driving task, such as a lane change or avoiding a parked car, that asks to run when its situation arises and changes the path it is given; several approved modules can run at once, in series.
Behavioural layer (behavioural decision making) (G-2912)	The level of planning between route planning and motion planning that decides which task the vehicle carries out now, such as following the lane, stopping at a line or giving way, from the route and what it perceives; the motion planner then plans that task.
Autonomy stack (G-3210)	The whole of a robot's software arranged in layers, sensing at the bottom and mission, behaviour, motion and control above it, each layer running at its own rate and working from the latest output of the layer above; the slow layers plan far ahead while the fast ones react, so a new reading reaches the wheels in one fast cycle.

Term	Meaning
Action (ROS 2) (G-3227)	A long job between two nodes, defined in an `.action` file: the client sends a goal, the server sends feedback while it works and a result when it ends, and the client can cancel, such as driving to a door pose; it is built from three services and two topics.